Eigenstate

Codebase Memory Engine

Reconstructing the "why" behind every engineering decision

Technical Proposal — Hackathon Submission

Team Name:	Team Pralay
Team Member 1:	*[Go Developer — Backend & Ingestion]*
Team Member 2:	*[Python Developer — AI Reasoning Engine]*
Team Member 3:	*[Frontend Developer — React UI]*

Hackathon:	*[NextGen]*
Track:	Developer Tools / AI Infrastructure
Date:	*[pta nhi bhai]*

Core Thesis: Git tells you *what* changed.
Eigenstate tells you *why it was worth the tradeoff*.

Confidential — Hackathon Submission Only

Contents

1 Abstract **3**

2 Problem Statement **3**
2.1 The Missing "Why" in Software Engineering . . . 3
2.2 Why Existing Tools Fail . . . 3
2.3 The Concrete Cost . . . 4

3 Proposed Solution **4**
3.1 Eigenstate: A Decision Reconstruction Engine . . . 4
3.1.1 Core Design Principles . . . 4
3.1.2 What a User Experiences . . . 4

4 System Architecture **5**
4.1 End-to-End Pipeline . . . 5
4.2 Component Responsibilities . . . 5
4.2.1 Go Ingestion Layer . . . 5
4.2.2 Python Intelligence Layer . . . 5
4.2.3 PostgreSQL Knowledge Store . . . 5
4.2.4 Go Query API . . . 5
4.2.5 React Frontend . . . 6
4.3 Data Flow Summary . . . 6

5 Detailed Technical Design **6**
5.1 Go Backend . . . 6
5.1.1 GitHub API Integration . . . 6
5.1.2 API Routes . . . 6
5.1.3 Data Ingestion Pipeline . . . 7
5.2 Python Intelligence Layer . . . 7
5.2.1 Text Preprocessing . . . 7
5.2.2 Diff-Based Inference . . . 8
5.2.3 Tree-sitter Function Extraction . . . 8
5.2.4 Confidence Scoring Engine . . . 9
5.2.5 Anti-Hallucination Prompt Design . . . 10
5.3 Database Design . . . 10
5.3.1 Schema . . . 10
5.3.2 Relationship Model . . . 12
5.4 Frontend Design . . . 12
5.4.1 UI Structure . . . 12
5.4.2 WHY Panel Components . . . 12
5.4.3 Lineage Visualisation . . . 12

6 Algorithm and Core Logic **13**
6.1 Multi-Signal Reasoning Pipeline . . . 13
6.2 Hallucination Reduction Strategy . . . 13
6.3 Confidence Scoring Logic . . . 14

6.4 Temporal Staleness Detection 14

7 Implementation Plan **14**
7.1 Role A — Go Developer: Backend & Ingestion 14
7.2 Role B — Python Developer: Intelligence Layer 15
7.3 Role C — Frontend Developer: React UI 15
7.4 Integration Timeline 16

8 Demo Flow **16**

9 Challenges and Mitigations **17**

10 Future Scope **18**
10.1 IDE Plugin 18
10.2 Real-Time PR Assistant 18
10.3 Enterprise Knowledge Graph Expansion 18
10.4 Decision Diff and Regression Detection 18
10.5 Onboarding Assistant 18

11 Conclusion **18**

1. Abstract

Engineering teams lose institutional memory every time a developer leaves, a ticket is closed, or a PR is merged with a one-line description. The reasoning behind architectural decisions — why a rate limiter was removed, why a validation layer was bypassed, why a particular tradeoff was accepted — disappears into the silence between lines of code.

Eigenstate is a decision reconstruction system that ingests a GitHub repository and builds a structured reasoning graph linking every function to the engineering decision that shaped it. Unlike code summarizers or AI chat assistants, Eigenstate is a **multi-signal reasoning engine**: it fuses commit messages, PR bodies, issue threads, and code diffs to infer intent with grounded, evidence-backed confidence scoring.

The result is a queryable knowledge layer over any codebase. A developer can click any function and receive a complete decision lineage: what changed, why it changed, what tradeoff was accepted, and how confident the system is — with every claim linked to its source.

2. Problem Statement

2.1 The Missing "Why" in Software Engineering

Every non-trivial codebase contains decisions that made perfect sense when they were made and are completely opaque twelve months later. This is not a tooling failure — it is a structural gap in how engineering knowledge is recorded and retrieved.

Consider the following scenario: a new engineer joins a team and finds a function `authMiddleware` that silently bypasses rate limiting for certain routes. The code works. The tests pass. But *why does this exist?*

- **Git log** shows: `fix: update middleware` — unhelpful.
- **PR #142** says: `merged 11 months ago` with no description.
- **The original author** has left the company.
- **Existing docs** are 18 months out of date.

The engineer now faces three bad choices: spend hours archaeologically tracing issues and Slack threads; make a guess and introduce a regression; or leave the code untouched and accumulate more technical debt.

2.2 Why Existing Tools Fail

- **GitHub Copilot / code assistants** are stateless. They explain what code *does*, not why a decision was made across multiple PRs over time.
- **Documentation tools** depend on developers writing documentation — the very behaviour that has already failed.

- **Code search** surfaces occurrences, not reasoning.
- **AI chat over codebase** hallucinates freely because it has no grounded evidence chain.

2.3 The Concrete Cost

Industry data consistently shows that engineers spend 30–40% of their time understanding existing code rather than writing new code. Poor decision provenance is a primary driver of slow onboarding, recurring bugs, and fear of refactoring. The problem is not talent — it is missing institutional memory.

> **The gap is not "code explanation." The gap is decision provenance.**
> What was decided? Why was it worth the tradeoff? Is that reasoning still valid today?

3. Proposed Solution

3.1 Eigenstate: A Decision Reconstruction Engine

Eigenstate is not a chatbot, a code summarizer, or a documentation generator. It is a **multi-signal reasoning reconstruction system** — a persistent knowledge layer that captures *why* code exists by fusing every available engineering signal and building a structured, queryable graph of decisions.

3.1.1 Core Design Principles

1. **No trust in single signals.** PR descriptions alone are often useless. Eigenstate combines commit messages, code diffs, issue bodies, and PR comments — using pattern detection and LLM inference together.
2. **Grounded reasoning only.** Every output is tagged with its evidence source. The system never presents a bare LLM inference as fact. Confidence levels (`HIGH`, `MEDIUM`, `LOW`, `INFERRED`) are computed deterministically before the LLM is consulted.
3. **Temporal awareness.** Decisions have timestamps. A reason that was valid in 2021 may be obsolete in 2024. Eigenstate detects when the same function has been modified after a decision was recorded and flags staleness explicitly.
4. **Zero dependency on developer behaviour.** The system works even when PRs are empty, commit messages are terse, and issues are closed without resolution. It infers from structural code changes when text signals are absent.
5. **Moat through accumulation.** Every new PR processed makes the knowledge graph richer. Over time, Eigenstate builds an organisation-specific memory that no stateless tool can replicate.

3.1.2 What a User Experiences

A developer opens Eigenstate, selects `authMiddleware`, and sees:

Decision:	Bypass rate limiting for internal service routes
Reason:	Internal services were being blocked under load; latency SLA violated
Tradeoff:	Reduced DDoS surface coverage in exchange for internal service reliability
Confidence:	HIGH — explicit keyword found in PR comment
Source:	PR #142, Issue #89, commit `a3f9c12`
Warning:	Function modified again in 2024 (PR #287) — verify reasoning still applies

4. System Architecture

4.1 End-to-End Pipeline

The Eigenstate architecture is a clean six-stage pipeline:

GitHub Repository ⟶ Go Ingestion Layer ⟶ Python Intelligence Layer ⟶ PostgreSQL Knowledge Store ⟶ Go Query API ⟶ React Frontend

4.2 Component Responsibilities

4.2.1 Go Ingestion Layer

Connects to the GitHub REST API and fetches all available signals for a target repository: pull requests (title, body, comments, linked files), commits (message, diff), and issues (title, body, labels). Data is normalised into a canonical internal format and written to a staging area in PostgreSQL. The Go layer is responsible for pagination, rate-limit handling, and incremental sync.

4.2.2 Python Intelligence Layer

Receives staged raw data and runs the multi-signal reasoning pipeline. This layer owns: text preprocessing, code diff parsing and pattern detection, Tree-sitter AST parsing for function boundary extraction, deterministic confidence scoring, LLM prompt construction and execution, and temporal staleness detection. It writes structured decision records back to PostgreSQL.

4.2.3 PostgreSQL Knowledge Store

Persists the full relationship graph: `functions` ↔ `pull_requests` ↔ `issues` ↔ `decisions`. Every decision record carries its evidence JSON, confidence metadata, and timestamp. Full-text search indexes enable fast retrieval by function name, file path, or keyword.

4.2.4 Go Query API

A lightweight REST API (Gin framework) that serves the frontend. Core endpoint: `GET /why?function={name}&repo={id}`. Returns the full decision record including lineage, evidence chain, timeline, and staleness warnings. Designed for sub-100ms response on cached queries.

4.2.5 React Frontend

A two-panel interface: a file and function explorer on the left, and a WHY panel on the right. The WHY panel renders the decision, reason, tradeoff, confidence badge, evidence sources, and the full temporal lineage as an interactive graph. Designed for engineers who want answers in under five seconds.

4.3 Data Flow Summary

1. User submits a GitHub repository URL.
2. Go ingestion fetches and stores all PRs, commits, issues, and diffs.
3. Python pipeline processes each PR: extracts functions touched via Tree-sitter, scores signal strength, calls LLM with grounded prompt, stores structured decision.
4. Frontend loads the function tree; user clicks a function.
5. Go API queries PostgreSQL, returns decision record with full lineage.
6. Frontend renders WHY panel with confidence, evidence, timeline, and staleness.

5. Detailed Technical Design

5.1 Go Backend

5.1.1 GitHub API Integration

The ingestion service uses the GitHub REST API v3. Key endpoints consumed:

- `GET /repos/{owner}/{repo}/pulls?state=closed&per_page=100` — all merged PRs
- `GET /repos/{owner}/{repo}/pulls/{pr}/files` — files changed per PR
- `GET /repos/{owner}/{repo}/pulls/{pr}/comments` — review comments
- `GET /repos/{owner}/{repo}/commits/{sha}` — commit detail with diff
- `GET /repos/{owner}/{repo}/issues/{number}` — linked issues

The Go service implements exponential backoff on rate-limit responses (`HTTP 429`) and writes raw payloads to a `raw_events` staging table before processing begins. This ensures the ingestion pipeline is restartable without re-fetching data from GitHub.

5.1.2 API Routes

Listing 1: Core API route definitions

```
// Ingestion trigger
POST   /api/v1/repos           -- register and begin ingestion
GET    /api/v1/repos/:id/status -- ingestion progress

// Query layer
GET    /api/v1/why             -- ?function=name&repo=id
```

```
GET    /api/v1/lineage        -- ?function=name&repo=id (full
   graph)
GET    /api/v1/functions      -- list all functions in repo
GET    /api/v1/timeline       -- ?function=name&repo=id
```

5.1.3 Data Ingestion Pipeline

Listing 2: Ingestion worker pattern

```go
type IngestJob struct {
    RepoID   string
    PRNumber int
}

func (w *Worker) ProcessPR(job IngestJob) error {
    pr, err := w.github.GetPR(job.RepoID, job.PRNumber)
    if err != nil { return err }

    files, _     := w.github.GetPRFiles(job.RepoID, job.PRNumber
       )
    comments, _ := w.github.GetPRComments(job.RepoID, job.
       PRNumber)
    issue, _     := w.github.GetLinkedIssue(pr)

    raw := RawEvent{
        PR:       pr,
        Files:    files,
        Comments: comments,
        Issue:    issue,
    }
    return w.store.SaveRawEvent(raw)
}
```

5.2 Python Intelligence Layer

5.2.1 Text Preprocessing

Before LLM inference, all available text signals are normalised and concatenated into a canonical input document:

Listing 3: Signal combination

```python
def build_signal_document(pr: dict, issue: dict) -> dict:
    return {
        "pr_title":     pr.get("title", ""),
        "pr_body":      pr.get("body", "") or "",
        "comments":     "\n".join(c["body"] for c in pr.get("
           comments", [])),
        "commit_msg":   pr.get("commit_message", ""),
        "issue_title":  issue.get("title", "") if issue else ""
           ,
        "issue_body":   issue.get("body", "") if issue else "",
```

```
        "diff_before":  pr.get("diff_before", ""),
        "diff_after":   pr.get("diff_after", ""),
        "pattern":      detect_pattern_change(
                            pr.get("diff_before", ""),
                            pr.get("diff_after", ""))
    }
```

5.2.2 Diff-Based Inference

Code diffs are parsed to detect structural intent changes — not just textual differences. Known pattern pairs signal common engineering decisions even when no text explanation exists:

Listing 4: Pattern detection engine

```
KNOWN_PATTERNS = {
    ("rateLimit", "skip"):      "rate limiting bypass",
    ("validate", "bypass"):     "validation removal",
    ("cache", "nocache"):       "caching disabled",
    ("auth.required", "public"):"auth gate removed",
    ("retry", "noretry"):       "retry logic removed",
    ("sync", "async"):          "synchronous to async migration
       ",
}

def detect_pattern_change(before: str, after: str) -> Optional[
   str]:
    bl, al = before.lower(), after.lower()
    for (pat_b, pat_a), description in KNOWN_PATTERNS.items():
        if pat_b in bl and pat_a in al:
            return description
    return None
```

5.2.3 Tree-sitter Function Extraction

File-to-function mapping uses Tree-sitter for AST-accurate boundary detection across JavaScript, TypeScript, Python, and Go source files. This avoids fragile regex-based heuristics and correctly handles nested functions, arrow functions, and method definitions.

Listing 5: Tree-sitter boundary extractor

```
from tree_sitter import Language, Parser
import tree_sitter_javascript as tsjs

JS_LANG = Language(tsjs.language())

QUERY = """
(function_declaration name: (identifier) @name) @fn
(method_definition   name: (property_identifier) @name) @fn
(variable_declarator
  name: (identifier) @name
  value: [(arrow_function)(function_expression)]) @fn
```

```
"""

def extract_functions(source: str) -> list[FunctionBoundary]:
    parser = Parser(JS_LANG)
    tree   = parser.parse(bytes(source, "utf-8"))
    query  = JS_LANG.query(QUERY)
    caps   = query.captures(tree.root_node)
    return [
        FunctionBoundary(
            name=name.text.decode(),
            start_line=fn.start_point[0] + 1,
            end_line=fn.end_point[0] + 1)
        for fn, name in zip(caps.get("fn",[]), caps.get("name"
            ,[]))
    ]
```

5.2.4 Confidence Scoring Engine

Confidence is computed *deterministically* before the LLM is invoked. This ensures that the confidence level is always grounded in observable evidence and is never a post-hoc LLM self-assessment:

Listing 6: Deterministic confidence engine

```
EXPLICIT_KEYWORDS = [
    "because", "in order to", "the reason", "due to",
    "to prevent", "this fixes", "trade-off", "tradeoff",
    "we decided", "the issue was", "this allows"
]

def compute_confidence(signals: dict) -> SignalStrength:
    all_text = " ".join([
        signals["pr_body"], signals["comments"],
        signals["commit_msg"], signals["issue_body"]
    ]).lower()

    # Tier 1: Explicit causal language found
    for kw in EXPLICIT_KEYWORDS:
        if kw in all_text:
            return SignalStrength("HIGH", 90,
                f"explicit keyword '{kw}' found", "pr_body or
                    comment")

    # Tier 2: Structural code pattern change
    if signals.get("pattern"):
        return SignalStrength("MEDIUM", 65,
            f"pattern: {signals['pattern']}", "code_diff")

    # Tier 3: Issue linked with content
    if signals["issue_body"] and len(signals["commit_msg"]) >
        20:
        return SignalStrength("LOW", 40,
```

```
            "issue context + commit message combined", "issue +
                commit")

    # Tier 4: Fully inferred
    return SignalStrength("INFERRED", 20,
        "no explicit reasoning; diff-only inference", "
           code_diff")
```

5.2.5 Anti-Hallucination Prompt Design

The LLM is given strict grounding rules. It may not invent evidence, and must cite which signal supports each claim. If it cannot reconstruct a decision with at least `LOW` confidence from the provided signals, it must return a structured null response rather than fabricate:

Listing 7: Core LLM system prompt (truncated)

```
You are Eigenstate -- a decision reconstruction engine.
You infer engineering intent ONLY from the evidence provided.

RULES (never violate):
1. Only claim what the evidence directly supports.
2. If reason is unclear, return null -- do NOT invent.
3. Cite the source signal for every field.
4. Tradeoffs must be specific -- not generic buzzwords.
5. Confidence is already computed. Do not reassess it.

Return ONLY valid JSON in this exact schema:
{
  "decision":          "string or null",
  "reason":            "string or null",
  "tradeoff":          "string or null",
  "evidence": {
    "primary":         "strongest signal + quote",
    "supporting":      ["signal 2", "signal 3"],
    "conflicts":       "any contradictions or null"
  },
  "suggested_pr_desc": "string -- what dev should have written"
}
```

5.3 Database Design

5.3.1 Schema

Listing 8: Core PostgreSQL schema

```
CREATE TABLE functions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repo_id     UUID NOT NULL,
  name        TEXT NOT NULL,
  file_path   TEXT NOT NULL,
  start_line  INT,
  end_line    INT,
```

```sql
  language     TEXT,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE pull_requests (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repo_id        UUID NOT NULL,
  github_pr_id   INT NOT NULL,
  title          TEXT,
  body           TEXT,
  merged_at      TIMESTAMPTZ,
  author         TEXT,
  UNIQUE(repo_id, github_pr_id)
);

CREATE TABLE issues (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repo_id          UUID NOT NULL,
  github_issue_id  INT NOT NULL,
  title            TEXT,
  body             TEXT,
  closed_at        TIMESTAMPTZ,
  UNIQUE(repo_id, github_issue_id)
);

CREATE TABLE decisions (
  id                 UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  function_id        UUID REFERENCES functions(id),
  pr_id              UUID REFERENCES pull_requests(id),
  issue_id           UUID REFERENCES issues(id),
  decision_text      TEXT NOT NULL,
  reason_text        TEXT,
  tradeoff_text      TEXT,
  confidence_level   TEXT CHECK (confidence_level IN
    ('HIGH','MEDIUM','LOW','INFERRED','INSUFFICIENT_DATA')),
  confidence_score   INT  CHECK (confidence_score BETWEEN 0 AND
     100),
  confidence_reason  TEXT,
  evidence           JSONB,
  suggested_pr_desc  TEXT,
  decision_ts        TIMESTAMPTZ NOT NULL,
  is_stale           BOOLEAN DEFAULT FALSE,
  staleness_warning  TEXT,
  model_used         TEXT,
  created_at         TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_decisions_function ON decisions(function_id);
CREATE INDEX idx_decisions_ts       ON decisions(decision_ts);
CREATE INDEX idx_decisions_fts      ON decisions
  USING GIN(to_tsvector('english',
```

```
    decision_text || ' ' || COALESCE(reason_text, ''));
```

5.3.2 Relationship Model

The graph structure is: **Function** → **Decision** ← **Pull Request** ↔ **Issue**. A single function may have multiple decisions across time (one per PR that touched it), forming a temporal lineage. The `decisions.evidence` JSONB column stores the full evidence chain without a rigid schema, allowing signal sources to vary per record.

5.4 Frontend Design

5.4.1 UI Structure

The React interface is a two-panel layout:

- **Left panel — File explorer:** Tree view of the repository file structure. Expandable directories, clickable files, and individual function entries. Functions with `HIGH` confidence decisions are badged in teal; `INFERRED` in amber; stale in orange.
- **Right panel — WHY panel:** Triggered on function click. Displays decision, reason, tradeoff, confidence badge, full evidence chain with source links, the suggested PR description, and a timeline of all decisions affecting the function.

5.4.2 WHY Panel Components

Listing 9: WHY panel data structure (React JSX)

```
const WhyPanel = ({ decision }) => (
  <div className="why-panel">
    <ConfidenceBadge level={decision.confidence_level}
                     score={decision.confidence_score} />
    <DecisionBlock   text={decision.decision_text} />
    <ReasonBlock     text={decision.reason_text} />
    <TradeoffBlock   text={decision.tradeoff_text} />
    <EvidenceChain   evidence={decision.evidence} />
    <SourceLinks     pr={decision.pr} issue={decision.issue} />
    {decision.is_stale && (
      <StalenessWarning warning={decision.staleness_warning} />
    )}
    <Timeline        events={decision.timeline} />
    <SuggestedPRDesc text={decision.suggested_pr_desc} />
  </div>
);
```

5.4.3 Lineage Visualisation

The timeline component renders a vertical lineage graph showing all PRs that touched a function in chronological order, with each node coloured by confidence level and linked to the decision record. This gives engineers an immediate visual sense of how a function has evolved and which decisions are recent versus historical.

6. Algorithm and Core Logic

6.1 Multi-Signal Reasoning Pipeline

The reasoning engine fuses four independent signal sources. Each is processed independently before being combined for LLM inference:

1. **PR text signals** (title + body + comments): scanned for explicit causal language. Presence of keywords such as *because*, *due to*, *tradeoff*, *the reason* immediately elevates confidence to `HIGH`.
2. **Commit message**: analysed for intent-indicating verbs (*fix*, *remove*, *revert*, *bypass*) and length heuristics. A commit message longer than 20 characters contributes to `LOW` confidence when combined with issue context.
3. **Code diff**: before/after comparison using known pattern pairs (see Section 5.2.2). Structural changes convey intent even when no text explanation exists. A change from `if (rateLimit) block()` to `if (internalService) skip()` unambiguously signals a bypass decision.
4. **Issue body**: provides problem context. A linked issue titled "Internal services timing out under load" directly supports a decision about rate limiting even if the PR itself is silent.

The combined document is passed to the LLM *only after* confidence is scored deterministically. The LLM's role is to synthesise a human-readable explanation — not to assess evidence quality.

6.2 Hallucination Reduction Strategy

Eigenstate reduces hallucination through three independent mechanisms:

1. **Pre-LLM confidence scoring:** The system computes confidence before the LLM is called. If confidence is `INSUFFICIENT_DATA`, the LLM is not called at all. This eliminates the most dangerous failure mode: confident hallucinations on empty input.
2. **Grounded prompt design:** The prompt explicitly forbids invention. Every output field must cite its source signal. The LLM is instructed to return structured null values rather than fabricate when evidence is absent.
3. **Evidence display to users:** Every decision shown in the UI includes its evidence chain. Engineers can inspect the primary and supporting signals themselves, making hallucinations immediately detectable.

6.3 Confidence Scoring Logic

Level	Score	Trigger Condition
HIGH	85–100	Explicit causal keyword in PR text or comments
MEDIUM	55–84	Known code pattern change detected in diff
LOW	30–54	Issue linked + commit message has meaningful content
INFERRED	10–29	Diff-only inference; no text signals available
INSUF. DATA	0–9	Too little signal; LLM not called; record flagged

6.4 Temporal Staleness Detection

Every decision is timestamped at the `merged_at` date of its originating PR. On query, the system checks whether the same function was modified by any *later* PR. If so, and if the original decision is older than a configurable threshold (default: 2 years), the system sets `is_stale = true` and generates a human-readable staleness warning citing the most recent PR that touched the function. This implements the "history of truth" model: decisions are not presented as eternal facts but as time-bounded records.

7. Implementation Plan

The build is divided across three roles. Each role has clear ownership, concrete deliverables, and defined interfaces with the other roles.

7.1 Role A — Go Developer: Backend & Ingestion

Phase 1: GitHub Ingestion (Hours 1–3)

- Set up Go project with Gin framework and `google/go-github` client
- Implement `GET /repos` registration endpoint and repo metadata store
- Build paginated PR fetcher with rate-limit backoff
- Fetch PR files, comments, linked commits, and linked issues
- Write raw JSON payloads to `raw_events` PostgreSQL table

Phase 2: Data Pipeline (Hours 3–5)

- Implement ingestion worker queue (simple channel-based, not message broker)
- Parse GitHub diff format: extract `before`/`after` hunks per file
- Write normalised records to `pull_requests`, `issues`, `functions` tables
- Emit job completion events to trigger Python processing

Phase 3: Query API (Hours 5–7)

- Implement `GET /api/v1/why` with function name + repo ID lookup

- Implement `GET /api/v1/lineage` returning ordered decision timeline
- Implement `GET /api/v1/functions` for file-tree endpoint
- Add CORS middleware for frontend

Tools: Go 1.22, Gin, `google/go-github`, `lib/pq`, PostgreSQL driver
Key Output: Working ingestion pipeline + REST API with seeded demo data

7.2 Role B — Python Developer: Intelligence Layer

Phase 1: Signal Processing (Hours 1–3)

- Set up FastAPI service, SQLAlchemy, `tree-sitter` + language bindings
- Build `build_signal_document()` combining all text signals
- Implement pattern change detector for known diff pairs
- Build Tree-sitter AST extractor for JS/TS/Python/Go
- Implement `map_diff_lines_to_functions()` using line range overlap

Phase 2: Reasoning Engine (Hours 3–5)

- Build deterministic confidence scorer (5-tier system)
- Write anti-hallucination LLM prompt with strict JSON schema
- Integrate Anthropic Claude API (or OpenAI) for extraction
- Validate LLM JSON output; handle `INSUFFICIENT_DATA` path
- Build staleness checker using decision timestamp vs. later PRs

Phase 3: Pipeline Integration (Hours 5–7)

- Implement `POST /process` endpoint consumed by Go ingestion
- Write completed decision records to PostgreSQL `decisions` table
- Pre-process selected demo repo (e.g. Fastify or Next.js) to seed data
- Build suggested PR description generator for the UI

Tools: Python 3.12, FastAPI, `tree-sitter`, `tree-sitter-javascript`, Claude/OpenAI API, SQLAlchemy, psycopg2
Key Output: Working reasoning engine with seeded decisions for demo repo

7.3 Role C — Frontend Developer: React UI

Phase 1: Layout and File Explorer (Hours 1–3)

- Set up React + Vite, Tailwind CSS, Axios
- Build two-panel layout: file explorer (left) + WHY panel (right)
- Implement recursive file tree component from `/api/v1/functions`
- Add confidence-level colour badges on function nodes
- Wire click handler to fetch WHY data for selected function

Phase 2: WHY Panel (Hours 3–5)

- Build `ConfidenceBadge` component (colour-coded, score shown)
- Build `DecisionBlock`, `ReasonBlock`, `TradeoffBlock` display cards
- Build `EvidenceChain` component listing source signals with links to GitHub
- Build `StalenessWarning` banner for stale decisions

- Build `SuggestedPRDesc` panel showing what the PR *should* have said

Phase 3: Timeline + Polish (Hours 5–7)

- Build vertical lineage timeline using `/api/v1/lineage`
- Add search bar to query functions by name or keyword
- Polish loading states, error states, empty states
- Prepare demo walkthrough: pre-select demo repo and function

Tools: React 18, Vite, Tailwind CSS, Axios, Recharts (timeline)
Key Output: Polished two-panel UI with working WHY panel and demo-ready lineage view

7.4 Integration Timeline

Milestone	Time	Deliverable
Ingestion live	Hour 3	PRs + diffs stored in DB
Reasoning engine live	Hour 5	Decisions written for demo repo
API live	Hour 5	`/why` endpoint returning data
UI connected	Hour 6	WHY panel showing real decisions
Demo rehearsal	Hour 7	Full flow end-to-end
Buffer + polish	Hour 8	Fallback: pre-loaded static data

8. Demo Flow

The demo is designed to land the core insight in under three minutes. Every step is scripted and every API call is pre-warmed.

1. **Open Eigenstate.** A pre-loaded repository (e.g. Fastify v4) is already ingested. The file tree is visible. The presenter states: *"This repo has 3 years of engineering decisions we've never been able to query. Until now."*

2. **Click a function.** The presenter clicks `authPreHandler` in the file explorer. The WHY panel animates in. Judges see: a decision, a reason, a tradeoff, and a confidence badge in under one second.

3. **Show the evidence.** The presenter expands the evidence chain. The primary signal is highlighted — a PR comment from 14 months ago: *"rate limiter was killing internal health checks."* The issue is linked. The commit diff is visible.

4. **Show the lineage.** The presenter scrolls to the timeline. Three decisions affecting this function over three years are visible in chronological order. The most recent shows a staleness warning.

5. **Show the bad PR.** The presenter clicks "View original PR." The PR description says: `fix: update middleware`. The presenter says: *"That's the PR. Eigenstate reconstructed the full decision from code diff and issue context — even though the developer wrote nothing."*

6. **Show the suggested description.** The UI shows the *Suggested PR description*: *"Removed rate limiting for internal service routes to fix latency SLO breach. Tradeoff: reduced DDoS coverage on internal endpoints."* The presenter says: *"This is also a tool for improving engineering culture going forward."*

7. **Close with the thesis.** *"Git tells you what changed. Eigenstate tells you why it was worth the tradeoff. This is engineering memory — queryable, grounded, and never lost."*

9. Challenges and Mitigations

1. **Missing or empty PR descriptions.**
   Challenge: A large proportion of real-world PRs contain minimal text.
   Mitigation: Multi-signal reasoning explicitly degrades gracefully. Code diff pattern detection provides `MEDIUM` confidence even on empty PRs. The system never withholds a result — it flags the confidence level and evidence source, leaving the engineer to judge.

2. **LLM hallucination.**
   Challenge: LLMs will invent plausible-sounding reasoning when given insufficient input.
   Mitigation: Three-layer defence: (1) pre-LLM confidence gating — LLM not called for `INSUFFICIENT_DATA`; (2) grounded prompt with explicit prohibition on invention; (3) evidence chain displayed to user so fabrications are detectable.

3. **Accurate function-to-file mapping.**
   Challenge: Regex-based file matching is brittle and misses nested functions, arrow functions, and renamed methods.
   Mitigation: Tree-sitter provides AST-accurate function boundary extraction with multi-language support. Diff line numbers are mapped to function ranges via direct line overlap, not heuristics.

4. **Stale knowledge.**
   Challenge: A decision from 2020 may be actively harmful to surface in 2025 if the context has changed.
   Mitigation: Temporal reasoning layer timestamps every decision and compares against subsequent PRs touching the same function. Staleness warnings are surfaced prominently in the UI, and the full timeline is always visible.

5. **Data privacy and code confidentiality.**
   Challenge: Enterprises cannot send proprietary code and internal decision history to third-party APIs.
   Mitigation: Eigenstate is designed for self-hosted deployment. The Python reasoning layer can use locally-hosted open-source models (Mistral, Llama) with no external API calls. The architecture is identical; only the model endpoint changes.

6. **Competitive threat from existing tools.**
   Challenge: GitHub Copilot and similar tools may expand into this space.
   Mitigation: Copilot is stateless. Eigenstate's moat is the *accumulated knowledge graph* — an org-specific memory that grows richer with every ingested PR and cannot be

replicated by a stateless inference call. Cross-PR reasoning, decision lineage, and temporal tracking are structurally impossible in a stateless model.

10. Future Scope

10.1 IDE Plugin

A VS Code extension would surface WHY panel data inline via a hover tooltip on any function. The engineer would never need to leave their editor. The extension makes a single `GET /why` call and renders the decision, confidence badge, and staleness warning as a CodeLens annotation.

10.2 Real-Time PR Assistant

A GitHub Actions integration that runs on PR open events. For every function modified in the PR, Eigenstate retrieves the existing decision record and posts a structured comment: *"This function was last modified in PR #142 with the following reasoning... Confirm this PR is consistent with or intentionally supersedes that decision."*

10.3 Enterprise Knowledge Graph Expansion

At scale, the knowledge graph can be enriched beyond GitHub: Jira tickets, Confluence pages, Slack threads, architecture review documents, and incident postmortems can all be ingested as additional signal sources. The confidence engine is signal-agnostic; new sources raise scores without changing the architecture.

10.4 Decision Diff and Regression Detection

When a new PR is ingested that touches a function with a `HIGH` confidence decision, Eigenstate can automatically check whether the new change *contradicts* the recorded reasoning. If it does, a regression warning is surfaced before merge — turning institutional memory into a live quality gate.

10.5 Onboarding Assistant

For new engineers, Eigenstate can generate a *decision map* of any module: the five most impactful decisions, the five most stale ones, and the three areas with `INSUFFICIENT_DATA` that need documentation attention. This transforms onboarding from "read all the PRs" to "here is what actually matters and why."

11. Conclusion

Software engineering has a memory problem. The decisions that shape a codebase — the tradeoffs accepted, the constraints understood, the problems solved — dissolve within months into uncommitted knowledge, departed colleagues, and terse commit messages.

Every team rebuilds this context from scratch with every onboarding, every refactor, every debugging session.

Eigenstate solves this with a system built on engineering rigour: multi-signal reasoning that works even when developers write nothing, grounded confidence scoring that refuses to hallucinate, and a temporal model that distinguishes living decisions from obsolete ones. It is not another AI assistant layered over a codebase. It is a decision reconstruction engine — the first system to treat engineering reasoning as a first-class, queryable, persistent artefact.

The technical foundation is sound: Go for performance-critical ingestion and API serving, Python for intelligence and LLM integration, Tree-sitter for language-accurate code analysis, and PostgreSQL for a knowledge graph that compounds in value with every new PR processed. The architecture is production-deployable, self-hostable, and extensible.

You are not explaining code.

You are preserving engineering memory.

The best engineering teams do not just write good code. They write code whose reasoning survives them. Eigenstate makes that survivability the default — for every team, every codebase, every decision that was ever worth making.

Eigenstate — Hackathon Technical Proposal — Team Eigenstate